第17小时

云原生架构设计理论与实践

17.0　章节考点分析

第 17 小时主要学习云原生架构设计理论与实践。根据考试大纲，本小时知识点会涉及单选题型（约占 2～4 分）、案例题（25 分）和论文题，本小时节内容偏重于方法掌握和应用，根据以往全国计算机技术与软件专业技术资格（水平）考试的出题规律，概念知识的考查内容多数来源于实际应用，还需要灵活运用相关知识点。

本小时知识架构如图 17.1 所示。

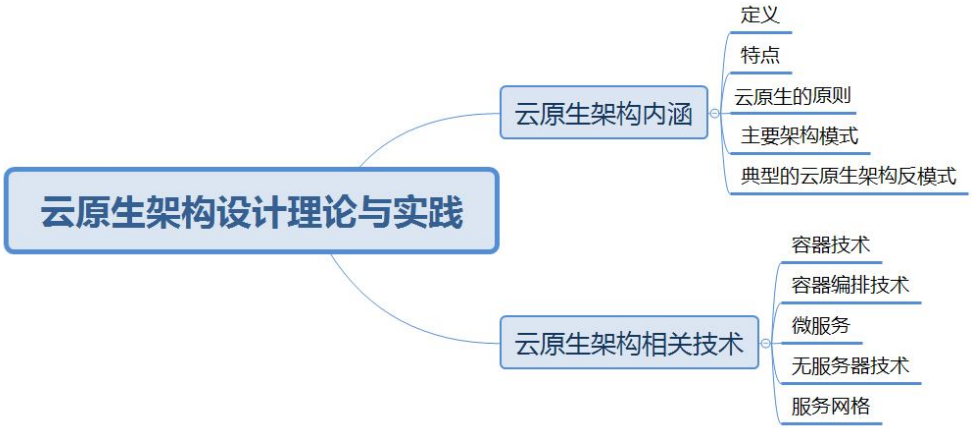

图 17.1　本小时知识架构

【导读小贴士】

云原生（Cloud Native）是近几年云计算领域炙手可热的话题，云原生技术已成为驱动业务增长的重要引擎。同时，作为新型基础设施的重要支撑技术，云原生也逐渐在人工智能、大数据、边缘计算、5G 等新兴领域崭露头角。伴随各行业上云的逐步深化，云原生化转型进程将进一步加速。

17.1 云原生架构内涵

【基础知识点】

1. 定义

云原生架构是基于云原生技术的一组架构原则和设计模式的集合，旨在将云应用中的非业务代码部分进行最大化地剥离，从而让云设施接管应用中原有的大量非功能特性（如弹性、韧性、安全、可观测性、灰度等），使业务不再有非功能性业务中断困扰的同时，具备轻量、敏捷、高度自动化的特点。

2. 特点

基于云原生架构的应用特点包括：

（1）代码结构发生巨大变化：不再需要掌握文件及其分布式处理技术，不再需要掌握各种复杂的网络技术，简化让业务开发变得更敏捷、更快速。

（2）非功能性特性大量委托给云原生架构来解决：比如高可用能力、容灾能力、安全特性、可运维性、易用性、可测试性、灰度发布能力等。

（3）高度自动化的软件交付：基于云原生的自动化软件交付可以把应用自动部署到成千上万的节点上。

3. 云原生的原则

云原生具有以下原则：

（1）服务化原则：通过服务化架构把不同生命周期的模块分离出来，分别进行业务迭代。

（2）弹性原则：弹性是指系统的部署规模可以随着业务量的变化而自动伸缩。

（3）可观测原则：通过日志、链路跟踪和度量等手段，使得多次服务调用的耗时、返回值和参数都清晰可见。

（4）韧性原则：软件所依赖的软硬件组件出现各种异常时，软件表现出来的抵御能力。

（5）所有过程自动化原则：让自动化工具理解交付目标和环境差异，实现整个软件交付和运维的自动化。

（6）零信任原则：不应该信任网络内部和外部的任何人/设备/系统，需要基于认证和授权重构访问控制的信任基础。

（7）架构持续演进原则：架构具备持续演进能力。

4. 主要架构模式

云原生涉及的主要架构模式有：

（1）服务化架构模式：要求以应用模块为颗粒度划分一个应用软件，以接口契约（例如 IDL）定义彼此业务关系，以标准协议（HTTP、gRPC 等）确保彼此的互联互通，结合领域模型驱动（Domain Driven Design，DDD）、测试动开发（Test Driven Design，TDD）、容器化部署提升每个接口的代码质量和迭代速度。

（2）Mesh 化架构模式：Mesh 化架构是把中间件框架（如 RPC、缓存、异步消息等）从业务进程中分离，让中间件 SDK 与业务代码进一步解耦，从而使得中间件升级对业务进程没有影响，甚至迁移到另外一个平台的中间件也对业务透明。

（3）Serverless 模式：业务流量到来/业务事件发生时，云会启动或调度一个已启动的业务进程进行处理，处理完成后云自动会关闭/调度业务进程，等待下一次触发。开发者不用关心应用运行地点、操作系统、网络配置、CPU 性能等，将应用的整个运行都委托给云。Serverless 模式适合事件驱动的数据计算任务、计算时间短的请求/响应应用、没有复杂相互调用的长周期任务。

（4）存储计算分离模式：分布式环境中的 CAP 困难主要是针对有状态应用，由于一致性（Consistency，C），可用性（Availability，A），分区容错性（Partition Tolerance，P）三者无法同时满足，最多满足其中两个。所以无状态应用不存在一致性这个维度，可以获得很好的可用性和分区容错性，因而获得更好的弹性。

（5）分布式事务模式。由于业务需要访问多个微服务，所以会带来分布式事务问题，否则数据就会出现不一致。因此架构师需要根据不同的场景选择合适的分布式事务模式，常用的有：

- XA 模式（传统采用 XA 模式）：由于 XA 规范是实现分布式事务处理的标准，通常采用两阶段提交（2 Prepare Commit，2PC）的方法，具有很强的一致性，但是由于需要两次网络交互，所以性能差。
- 基于消息的最终一致性（BASE）：在可用性和一致性相冲突的情况下，为了权衡二者，BASE 提出只要满足基本可用（BA）和最终一致性（E），接受数据的软状态或未确定状态（S），来优先实现性能，所以这类系统通常具备很高的性能。但正是由于应用的特点，选择可用性和一致性的妥协方案，导致通用性有限。
- TCC 模式：采用 Try-Confirm-Cancel 二阶段模式，事务隔离性可控，高效，但需要应用代码将业务模型拆成二阶段，所以对业务侵入性强，设计开发维护等成本很高。
- SAGA 模式：每个正向事务都对应一个补偿事务，若正向事务执行失败，则会执行补偿事务进行回滚。所以开发维护成本高。
- 开源项目 SEATA 的 AT 模式：它将 TCC 模式中的二阶段委托给底层代码框架，并且取消了行锁，所以非常高性能且无代码开发工作量，且可以自动执行回滚操作，但存在一些使用场景限制。

（6）可观测架构：可观测架构包括 Logging、Tracing、Metrics，其中 Logging 提供多个级别

跟踪，例如 INFO/ DEBUG/WARNING/ERROR；Tracing 收集一个请求从前端到后端的访问日志聚合，形成完整调用链路跟踪；Metrics 则提供对系统量化的多维度度量，包括并发度、耗时、可用时长、容量等。

（7）事件驱动架构：事件驱动架构（Event Driven Architecture，EDA）是一种应用/组件间的集成架构模式。适用于增强服务韧性、数据变化通知、构建开放式接口、事件流处理、命令查询的责任分离（Command Query Responsibility Segregation，CQRS）把对服务状态有影响的命令用事件来发起，而对服务状态没有影响的查询才使用同步调用的 API 接口等。

5. 典型的云原生架构反模式

架构设计有时候需要根据不同的业务场景选择不同的方式，常见的云原生反模式有：

（1）庞大的单体应用：缺乏依赖隔离，代码耦合，责任和模块边界不清晰，模块间接口缺乏治理，变更影响扩散，不同模块间的开发进度和发布时间要求难以协调，一个子模块不稳定导致整个应用都变慢，扩容时只能整体扩容而不能对达到瓶颈的模块单独扩容等。

（2）单体应用“硬拆”为微服务：强行把耦合度高、代码量少的模块进行服务化拆分；拆分后服务的数据是紧密耦合的；拆分后成为分布式调用，严重影响性能。

（3）缺乏自动化能力的微服务：人均负责模块数上升，人均工作量增大，也增加了软件开发成本。

17.2 云原生架构相关技术

【基础知识点】

1. 容器技术

容器作为标准化软件基础单元，它将应用及其所有依赖项打包发布，由于依赖项齐备，应用不再受环境限制，在不同计算环境间快速、可靠地运行。容器部署模式与其他模式的比较如图 17.2 所示。

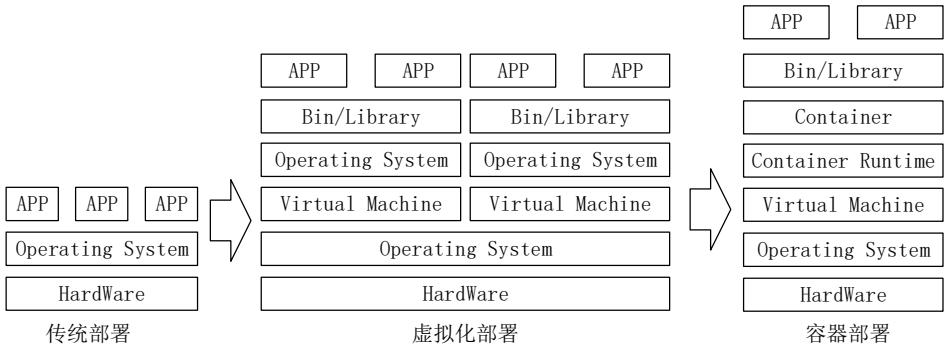

图 17.2 传统、虚拟化、容器部署模式的比较

2. 容器编排技术

容器编排技术包括资源调度、应用部署与管理、自动修复、服务发现与负载均衡、弹性伸缩、声明式 API、可扩展性架构、可移植性。

3. 微服务

微服务模式将后端单体应用拆分为松耦合的多个子应用，每个子应用负责一组子功能。这些子应用称为“微服务”，多个“微服务”共同形成了一个物理独立但逻辑完整的分布式微服务体系。这些微服务相对独立，通过解耦研发、测试与部署流程，提高整体迭代效率。微服务设计约束如下：

（1）微服务个体约束：微服务应用的功能在业务领域划分上应是相互独立的。

（2）微服务与微服务之间的横向关系：在合理划分好微服务间的边界后，从可发现性和可交互性处理微服务间的横向关系。可发现性是指当服务 A 发布和扩/缩容的时候，依赖服务 A 的服务 B 在不重新发布的前提下，能够自动感知到服务 A 的变化。可交互性是指服务 A 采用什么样的方式可以调用服务 B。

（3）微服务与数据层之间的纵向约束：提倡数据存储隔离（Data Storage Segregation，DSS）原则，对于数据的访问都必须通过相对应的微服务提供的 API 来访问。

（4）全局视角下的微服务分布式约束：高效运维整个系统，从技术上实现全自动化的 CI/CD 流水线满足对开发效率的诉求，并在这个基础上支持蓝绿、金丝雀等不同发布策略，以满足对业务发布稳定性的诉求。

4. 无服务器技术

无服务器技术的特点：

（1）全托管的计算服务——客户只需要编写代码构建应用，无须关注同质化的、负担繁重的基于服务器等基础设施的开发、运维、安全、高可用等工作。

（2）通用性——结合云 BaaS（后端云服务）API 的能力，能够支撑云上所有重要类型的应用。

（3）自动弹性伸缩——让用户无须为资源使用提前进行容量规划。

（4）按量计费——让企业的使用成本有效降低，无须为闲置资源付费。

无服务器技术的关注点是：计算资源弹性调度（容错、资源利用率、性能、数据驱动）、负载均衡和流控、安全性。

5. 服务网格（Service Mesh）

服务网格旨在将那些微服务间的连接、安全、流量控制和可观测等通用功能下沉为平台基础设施，实现应用与平台基础设施的解耦。图 17.3 展示了服务网格的典型架构。服务 A 调用服务 B 的所有请求，都被其下的服务代理截获，代理服务 A 完成到服务 B 的服务发现、熔断、限流等策略，而这些策略的总控是在控制平面（Control Plane）上配置。

服务网格的主要技术：Istio、Linkerd、Consul。

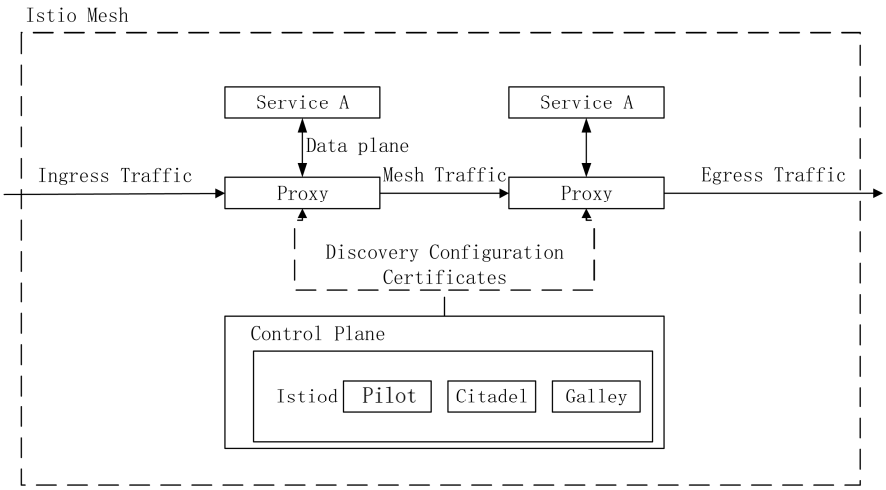

图 17.3　服务网格的典型架构

17.3　练习题

1．云的时代需要新的技术架构，来帮助企业应用能够更好地利用云计算优势，充分释放云计算的技术红利。云计算无法为企业带来的改进是（　　）。

A．通过 DevSecOps 应用开发模式，业务功能开发更加敏捷，提升迭代速度，成本更低

B．企业软件架构可以获得强大的可伸缩性和高可用性

C．结合云平台全方位企业级安全服务和安全合规能力，保障企业应用在云上安全构建，业务安全运行

D．企业的开发人员只须关注业务代码部分的开发，非业务功能可以完全委托给云原生架构来解决

解析：云原生架构旨在将云应用中的非业务代码部分进行最大化的剥离，从而让云设施接管应用中原有的大量非功能特性（如弹性、韧性、安全、可观测性、灰度等），但无法接管所有的非功能特性。

答案：D

2．下列关于云原生架构原则的描述，错误的是（　　）。

A．服务化原则、弹性原则、韧性原则

B．可观测原则、所有过程自动化原则

C．零信任原则、接口隔离原则

D．架构持续演进原则

解析：接口隔离原则是面向对象设计原则，其含义是使用多个专门的接口比使用单一的总接口

好。它不是云原生架构原则。

答案：C

3．关于微服务的描述，错误的是（　　）。

A．微服务是将后端单体应用拆分为松耦合的多个子应用，每个子应用负责一组子功能

B．微服务相对独立，通过解耦研发、测试与部署流程，提高整体迭代效率

C．微服务与数据层之间的纵向约束的含义是：在合理划分好微服务间的边界后，主要从微服务的可发现性和可交互性处理服务间的关系

D．驾驭微服务的前提是：高效运维整个系统，从技术上要准备全自动化的 CI/CD 流水线满足对开发效率的诉求，并在这个基础上支持蓝绿、金丝雀等不同发布策略

解析：在合理划分好微服务间的边界后，主要从微服务的可发现性和可交互性处理服务间的关系，是属于微服务之间的横向关系。正确的纵向约束是：对于微服务的私有数据的访问都必须通过当前微服务提供的 API 来访问。

答案：C

4．无服务器技术的特点之一是全托管的计算服务：客户只需要编写代码构建应用，无须关注同质化的、负担繁重的基于服务器等基础设施的（　　）等工作。

A．开发、测试、发布、交付

B．开发、运维、安全、高可用

C．机房建设、服务器装机、操作系统安装、软件安装

D．资源调度、性能压测、负载均衡、数据统计

解析：无服务器技术的特点如下。

全托管的计算服务：客户只需要编写代码构建应用，无须关注同质化的、负担繁重的基于服务器等基础设施的开发、运维、安全、高可用等工作。

通用性：结合云 BaaS API 的能力，能够支撑云上所有重要类型的应用。

自动弹性伸缩：让用户无须为资源使用提前进行容量规划。

按量计费：让企业的使用成本有效降低，无须为闲置资源付费。

答案：B

5．容器作为标准化软件单元，它将应用及其所有依赖项打包，使应用不再受（　　）限制，在不同计算环境间快速、可靠地运行。

A．环境　　B．操作系统　　C．硬件　　D．网络

解析：在容器的帮助下，应用程序无须关注操作系统及更加低层的硬件、网络、存储的限制。选项 B、C、D 的说法有局限性，选项 A 更贴切。

答案：A